Adaptive Cognitive Load Prediction System for Online Learning Using Behavioral Patterns

INDIVIDUAL THESIS

Madiththa Hene Rajitha Isuru Subasinghe

IT22089786

Individual component: Real-time Cognitive Load Prediction

Group ID: R26-SE-004

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science Honours in Information Technology

Specialisation in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

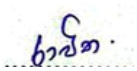
DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Subasinghe M.H.R.I

Student ID: IT22089786

Signature: 

Date: 11/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Dr. Kalpani Manathunga		
Co-supervisor: Mr. Jeewaka Perera		

ABSTRACT

Online learning platforms often track video activity without considering the mental effort experienced by learners. This study addresses this gap by developing and evaluating a cognitive load prediction component that uses video interactions without specialized sensors. The component forms part of an explainable adaptive learning platform.

The study involved 171 undergraduate computing students, including 108 laboratory and 63 remote participants, who watched two instructional videos. Data collection produced 1,420 valid two-minute observation windows. Six behavioral features were extracted. Such as pause frequency, forward navigation count, rewatch count, playback-rate changes, idle duration, and time on content. After each segment, students rated their mental effort from 1 to 9 using the Paas scale. These ratings were grouped into five cognitive load levels such as Very Low, Low, Medium, High, and Very High.

Logistic Regression, Decision Tree, XGBoost, and Random Forest were compared using an 80/20 participant-level split, ensuring that each participant's observations appeared only in training or testing. Random Forest achieved the strongest performance, with 79.52% accuracy, 79.48% balanced accuracy, and 79.55% macro F1, correctly classifying 233 of 293 test windows. The deployed pipeline combines median imputation with a 200-tree Random Forest classifier. A FastAPI service delivers predictions, stores results, forwards features for LIME explanations, and analyses recent cognitive load trends.

The findings support the feasibility of estimating cognitive load from video behavior within the studied context. However, subjective ratings, limited video diversity, dataset traceability, and differences in watch-time calculations constrain interpretation. The individual contribution is the behavioral prediction component and its empirical evaluation. Further research should examine its generalizability and effects on learning outcomes.

Keywords: cognitive load prediction, video learning, behavioral analytics, Random Forest, Paas mental effort

ACKNOWLEDGEMENTS

I thank Dr. Kalpani Manthunga for research supervision and constructive feedback, and Mr. Jeewaka Perera for technical guidance and support. I acknowledge Prof. Koliya Pulasinghe for coordination of the Research Project module and the guidance provided through the module.

I also thank my research group members Sanjaya Jayasinghe, Wimukthi Dilshan, and Imesh Liyanaarachchi for their collaboration on the shared platform. I thank Dr. Nilanga Abeysinghe for the advice on mental-effort measurement documented in the component evidence report. I thank the students who voluntarily contributed behavioural observations and ratings.

TABLE OF CONTENTS

ACKNOWLEDGEMENTS iii

1 INTRODUCTION 1

1.1 Overall project context 1

1.2 Individual component context 2

1.3 Problem statement 2

1.4 Research gap 3

1.5 Aim and objectives 3

1.6 Research questions or hypotheses 4

1.7 Scope limitations and assumptions 4

1.8 Individual contribution 5

1.9 Thesis structure 5

2 LITERATURE REVIEW AND RELATED WORK 7

2.1 Review method and scope 7

2.2 Domain and theoretical foundations 7

2.3 Related research 8

2.4 Existing systems products and techniques 9

2.5 Gap synthesis and conceptual framework 10

3 RESEARCH METHODOLOGY 11

3.1 Research design 11

3.2 Data participants or study materials 11

3.3 Experimental or study procedure 12

3.4 Measures and evaluation metrics 13

3.5 Data analysis 14

3.6 Ethics privacy security and responsible AI 15

3.7 Reproducibility and Research Artefacts 15
4 PRODUCT AND SOLUTION DESIGN 16
4.1 Requirements 16
4.2 Component architecture within the complete solution 17
4.3 Design alternatives and decisions 18
4.4 Detailed design 18
4.5 Standards and professional practice 19
5 IMPLEMENTATION INTEGRATION AND TESTING 20
5.1 Development of environment and technology stack 20
5.2 Component implementation 20
5.3 Integration with the other components 21
5.4 Verification and software testing 22
5.5 Deployment and operation 22
6 RESULTS AND VALIDATION 24
6.1 Research results 24
6.2 Product and technical results 26
6.3 User or stakeholder validation 27
6.4 Integration results 27
6.5 Summary of findings 27
7 DISCUSSION 29
7.1 Interpretation of findings 29
7.2 Comparison with literature baselines and existing solutions 29
7.3 Research contribution 30
7.4 Product contribution 30
7.5 Limitations and threats to validity 30
7.6 Implications 31

8 CONCLUSION AND FUTURE DIRECTIONS 33
8.1 Conclusion 33
8.2 Achievement of objectives 33
8.3 Individual research and product contributions 34
8.4 Future directions 35
REFERENCES 36
GLOSSARY 37
APPENDICES 38
Appendix A Research instruments and evaluation evidence 38
A.1 Collection instrument 38
A.2 Study figures and development export reconciliation 38
Appendix B Academic integrity and AI use disclosure 39

LIST OF FIGURES

Figure 1: Video-based collection interface and segment-rating prompt. 12
Figure 2:Cognitive load prediction workflow within the shared platform. 17
Figure 3:Reported Random Forest confusion matrix on 293 held-out windows. 25
Figure 4: Feature importance ranking in the supplied model evaluation. 26

LIST OF TABLES

Table 1:Six-feature inference contract 14
Table 2:Component requirements and implementation evidence 16
Table 3:Component test coverage 22
Table 4:Reported classifier comparison with percentage metrics 24
Table 5:Class recall calculated from the reported matrix 25

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial intelligence
API	Application programming interface
CLT	Cognitive Load Theory
CSV	Comma-separated values
F1	Harmonic mean of precision and recall
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LIME	Local Interpretable Model-agnostic Explanations
LLM	Large language model
ML	Machine learning
RF	Random Forest
SHAP	SHapley Additive exPlanations
UTC	Coordinated Universal Time
XAI	Explainable artificial intelligence

1 INTRODUCTION

1.1 Overall project context

The group project develops an explainable adaptive learning platform that combines cognitive load prediction, cognitive style profiling, grounded educational content processing, and instructional support. Its purpose is to make a learner's current interaction evidence available to teachers and to the components responsible for adapting explanations, activities, and lesson recommendations. Cognitive Load Theory provides the theoretical basis for this study. Processing unfamiliar material places demands on the limited capacity of working memory. The way learning material is presented can also create unnecessary processing demands [1]. An adaptive platform therefore needs timely information about the learning situation rather than relying only on completed assessments.

The overall system integrates behavioral sensing, explainable artificial intelligence, knowledge extraction, and controlled content generation into a learning loop. The cognitive load component estimates five cognitive load levels from video behavior. Other components interpret the supporting signals using Local Interpretable Model-agnostic Explanations **(LIME)** and SHapley Additive exPlanations **(SHAP),** detect cognitive styles, process teacher resources, and generate learning support under teacher review. These functions belong to the group project. This individual thesis focuses on the video interaction pipeline, mental effort labels, the deployed classifier, reliability checks, and cognitive load trend analysis.

The implementation uses a React interface, an Express API gateway, and separate Python and Node.js services. This architecture supports modular development and maintenance. The cognitive load prediction service provides learner-state information to the platform's explainability and adaptive learning components through clearly defined API interfaces.

1.2 Individual component context

The individual component is Real-Time Cognitive Load Prediction for video based online learning. It converts learner interactions into non overlapping two minute observation windows and classifies each suitable window as Very Low, Low, Medium, High, or Very High. The six inputs are pause frequency, forward navigation count, rewatch count, playback-rate change count, idle duration, and time on content. No webcam, eye tracker, electroencephalography sensor, or other biometric input is used by this classifier.

The label reference is the learner's self-reported mental effort, collected with the Paas nine-point scale [2]. The study maps the nine response values to five ordered categories. The classifier thus estimates a category of reported mental effort from observable video behavior. It does not directly measure all psychological dimensions of cognitive load or identify whether a learner is confused, fatigued, bored, or unmotivated.

1.3 Problem statement

A conventional learning platform can record video progress without providing a timely estimate of the mental effort associated with the learning segment. A completed video, a long viewing time, or repeated pauses is difficult to interpret in isolation. Teachers therefore lack a consistent way to distinguish a potentially demanding segment from a session that contains too little activity to support any interpretation.

The research problem is to develop and evaluate a non-intrusive classifier that uses ordinary video interaction logs to estimate five levels of reported mental effort and expose its output through a service suitable for an adaptive learning platform. The engineering problem includes segment alignment, feature consistency, insufficient activity, persistence, and delivery of evidence to downstream modules. A useful solution must also report its uncertainty and avoid presenting inactivity as a valid cognitive diagnosis.

1.4 Research gap

The reviewed studies offer complementary evidence about cognitive load, video behavior, physiological sensing, and adaptation. Merkt et al. connect pausing with comprehension difficulty and meaningful breakpoints, showing why a pause cannot be treated as a unique sign of overload [3]. Cai and Demmans Epp investigate cognitive load and affect through physiological sensing [4], while Ajayi and Avksentyeva explore eye-tracking-based classification [11]. These approaches motivate workload modelling but differ from browser-only video interaction service.

The practical gap addressed here is the integration of five-class behavioral inference with segment level self-report, explicit reliability handling, and accessible service output. The contribution is not a claim that behavioral measurement or Random Forest classification is new. It is a bounded implementation and evaluation of these ideas for undergraduate computing students, with a shared feature contract and a temporal view of saved predictions.

1.5 Aim and objectives

Aim: To develop and evaluate a non-intrusive, near-real-time cognitive load prediction component for undergraduate computing students engaged in video-based online learning.

Objectives

- Establish a consent based data collection procedure that links anonymous video interaction data with Paas mental effort ratings collected after each segment.
- Extract six consistent behavioral features from each two-minute observation window.
- Compare candidate classifiers using separate participant groups for training and testing and select a model suitable for deployment.
- Integrate prediction, reliability checks, data storage, and the transfer of features to explainability services through an API.
- Provide cognitive load trend and risk summaries and verify the service using automated tests.

Evaluation scope

- Classification metrics measure agreement between predictions and the reference labels.
- Service tests verify that the implementation behaves correctly.
- Further evaluation is required to determine whether generated learning support improves educational outcomes.

1.6 Research questions or hypotheses

1. Can six video interaction features distinguish five levels of self-reported mental effort among the students in the study?
2. Which classifier (Logistic Regression, Decision Tree, XGBoost, or Random Forest) achieves the highest accuracy and macro F1-score when participants are separated between training and testing?
3. Can the component deliver predictions through a responsive API while skipping observation windows with insufficient learning activity?
4. How can recent saved predictions be used to summarize cognitive load trends without training a separate temporal classifier?

These questions are evaluated as classification and engineering questions. The thesis does not test a causal hypothesis that the component reduces cognitive load, improves retention, or changes examination performance. Those outcomes require a separate intervention study with suitable controls.

1.7 Scope limitations and assumptions

The study scope is undergraduate computing students watching two selected instructional videos. The authoritative study account comprises 171 participants, including 108 laboratory and 63 remote participants, and 1,420 valid two-minute feature windows. The remote route extends collection beyond a single controlled room but introduces differences in devices, interruptions, and connectivity. Findings therefore relate to the recorded study conditions rather than all online learners or disciplines.

The service assumes correctly associated learner, lesson, session, and window identifiers. It also assumes that frontend feature calculation follows the training feature definitions. Mental-effort labels are subjective reference measurements, and repeated windows from one participant are correlated. The reported 80/20 split is at participant/session level; it must not be interpreted as an exact 80/20 division of windows.

1.8 Individual contribution

The individual contribution covers the cognitive load data collection protocol, six feature video representation, classification model integration, reliability gate, prediction endpoints, data persistence, and recent history trend analysis. The component supplies learner state and behavioral evidence to the shared platform. Its service interfaces enable explainability and adaptation without embedding those separate responsibilities inside the classifier.

1.9 Thesis structure

- **Chapter 2:** Reviews the theories and previous research relevant to cognitive load prediction.
- **Chapter 3:** Explains the participants, data collection, labelling, evaluation methods, ethics, and reproducibility.
- **Chapter 4:** Presents the component's requirements and design.
- **Chapter 5:** Describes implementation, integration, testing, and local operation.
- **Chapter 6:** Presents model performance, API response time, and integration results.
- **Chapter 7:** Discusses the findings, contributions, limitations, and implications.
- **Chapter 8:** Summarizes the conclusions, achievement of objectives, and future research.
- **References:** Lists all sources cited in the thesis using IEEE referencing style.

- **Glossary:** Defines specialized terms used in the thesis.
- **Appendices:** Include the data collection instrument, detailed evaluation counts, dataset-version explanations, and AI-use disclosure.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

The literature review examines previous research on cognitive load theory, mental-effort measurement, educational video interactions, physiological modelling, real-time clickstream processing, personalization, and machine learning classification. It follows a focused review approach rather than a systematic review.

Sources were selected based on their relevance to the measurement methods and implementation decisions of the cognitive load prediction component. Results from other studies are discussed in context rather than directly compared with this study's accuracy, as their participants, labels, input data, and evaluation methods differ.

2.2 Domain and theoretical foundations

Cognitive Load Theory relates instructional design to human cognitive architecture [1]. When learners process unfamiliar material, working memory has limited capacity. Content complexity and presentation demands interact with prior knowledge, making cognitive load dependent on the learner and task. The theory gives a rationale for monitoring demanding learning segments, but it does not imply one-to-one mapping between a mouse action and a psychological state.

Paas's work establishes a mental-effort rating as a practical subjective measurement in learning research [2]. The nine-point response scale provides an accessible segment-level reference without physiological equipment. In this project, the rating is requested after the two-minute segment, so the feature vector and response refer to the same recent experience. The five-class grouping is a project operationalization of the nine-point responses; it is not a separate instrument validated by Paas.

Self-report supports feasibility but introduces recall, interpretation, and response-style variation. The model predicts the grouped reference response. It cannot separately estimate intrinsic and extraneous load because the collected instrument does not independently label these constructs. Confidence from the classifier is likewise a model output rather than a psychological certainty measure.

2.3 Related research

Merkt et al. triangulate educational video logs, laboratory behavior, and video characteristics [3]. Their results associate pausing with perceived comprehension difficulty and meaningful structural breakpoints. This directly supports the relevance of pause frequency, while also showing its ambiguity. A learner may pause to take notes or regulate learning successfully. The present study consequently combines several behaviors instead of imposing a deterministic overload rule on pauses alone.

Cai and Demmans Epp describe modelling cognitive load and affect using an eye tracker and a wireless wristband [4]. Their work recognizes that workload changes alongside learning activities and that effect can influence interpretation. The present component shares interest in temporally contextualized learner modelling but uses browser interactions and self-reported labels. It does not reproduce their physiological measures or jointly model affect.

Ajayi and Avksentyeva compare supervised methods for cognitive load prediction with eye-tracking data, including a computer-generated dataset [11]. Their study helps motivate comparison of interpretable baselines and nonlinear classifiers, but its sensing modality and dataset construction differ from the present collection protocol. Its findings cannot establish performance for this component. The comparison in Chapter 6 is based on the project's own evaluation evidence.

Gayakwad et al. discuss a real-time clickstream lifecycle using Apache technologies, storage, and visualization [6]. The relevant lesson is that timely analytics require an organized ingestion and transformation pipeline. Their commercial behavior context does not validate cognitive load labels. This prototype adopts a simpler gateway, relational database, and window processing architecture because the implemented study does not demonstrate a need for a distributed streaming cluster.

Lange examines relationships between e-learning personalization and cognitive load [7]. The study supports considering learner control and instructional presentation in digital learning. However, an association between personalization and load is not proof that this platform's generated interventions improve learning. This distinction is important when interpreting group level expert ratings.

Random Forest combines multiple decision trees whose predictions are aggregated [8]. It is a suitable candidate for structured behavior features because relationships may be nonlinear and involve interactions. This is a methodological justification for evaluation, not a guarantee of superiority. Selection in this project follows the reported candidate comparison.

Paas investigated mental effort during learning using a nine-point self-report scale [2]. Learners use this scale to indicate how much mental effort they invested in a task, from very low to very high. This provides a practical reference measure without specialized equipment. In the present study, students completed the scale after each two-minute video segment. Their ratings were then grouped into five cognitive load levels for model training and evaluation. This five-class grouping is specific to the present study.

2.4 Existing systems products and techniques

Three broad approaches inform the design. Sensor based workload systems use physiological signals and can offer rich measurements but require sensing equipment or additional calibration. Behavioral learning analytics use activity logs already available in learning platforms, improving implementation accessibility but producing indirect and context dependent evidence. Adaptive systems use learner information to change content or support but need a reliable interface between sensing and instructional decisions.

The component follows the behavioral approach. Its six feature contract reduces sensing requirements and makes the input inspectable. The broader platform adds explainability and teacher reviewed support. LIME explains an individual prediction using a local surrogate [9]; SHAP represents additive feature contributions [10]. These explanations describe model behavior rather than establish the causal reason for a learner's mental effort. Preserving that distinction helps prevent pedagogical overinterpretation.

2.5 Gap synthesis and conceptual framework

The conceptual framework links a learning segment, observed video behavior, a self reported reference rating, an estimated load category, and downstream support. During collection, the rating supplies the supervised target. During deployment, the classifier estimates that target from behavior, subject to a reliability gate. The predictions saved can then be examined over time or supplied to explanatory services.

This framework addresses an operational gap between ordinary video logging and a usable cognitive state interface. Its validity depends on label quality, participant separated testing, consistency of feature definitions, and clearly communicated limitations. Five classes provide more detail than a binary label but also create difficult adjacent class distinctions. The results must therefore include class level recall and confusion counts rather than accuracy alone.

3 RESEARCH METHODOLOGY

3.1 Research design

The research combines system development with supervised classification evaluation. The developed system is a behavioral prediction service integrated with a video learning interface. The study uses video interaction data and Paas mental-effort ratings collected from undergraduate computing students after each two-minute learning segment. These observations support model training and evaluation, while software testing verifies the prediction service's functionality.

The study does not manipulate an intervention to establish learning gains. Instead, it evaluates agreement between predicted categories and grouped mental effort ratings, compares candidate algorithms, and verifies service behavior. Development proceeds through collection, preprocessing, model comparison, deployment, and software validation. The research and engineering outputs are reported separately to avoid equating a passing API test with classifier validity.

3.2 Data participants or study materials

The study involved 171 undergraduate computing students, comprising 108 laboratory and 63 remote participants. Participants watched two instructional videos. After data quality checks, 1,420 valid, non-overlapping two-minute feature windows were retained [2]. These windows represent repeated observations from participants rather than separate individuals. The evaluation confusion matrix contains 293 held-out test windows.

Participants got an anonymous code, selected a lesson, and started a learning session. They watched the video naturally and could pause, rewind, navigate, or change the playback speed. Every two minutes, a Paas rating prompt asked them to rate the mental effort experienced during that segment. Participation was voluntary, and the collected interaction data and ratings were used for academic research.

The consent material lists anonymous codes, session/video identifiers, pause events, navigation, rewatches, playback-speed changes, idle duration, time spent on content, and Paas ratings as collected data. The thesis does not infer demographic

distributions, recruitment criteria beyond the documented population, or formal ethics approval details that were not supplied.

3.3 Experimental or study procedure

Participants first received the purpose and privacy explanation and agreed to voluntary participation. The laboratory route used the assigned code and lesson recorded on the consent material. The alternative route allowed code generation and lesson selection.

Participants started a session and watched the video normally. They could pause, move forward, rewind, and alter playback speed as needed. The platform recorded the resulting behavior and aligned it with successive two minute segments. A Paas rating prompt appears every two minutes, asking participants to rate the mental effort they experienced during that segment.

The mental-effort response is kept separate from predictor features. Ratings 1–2 map to Very Low, 3–4 to Low, 5 to Medium, 6–7 to High, and 8–9 to Very High. This mapping produces target scores 1–5. Participant and session identifiers support linkage and grouped splitting. They are not model predictors. The response prompt belongs to collection and evaluation, whereas deployment estimates load without requiring a rating after every prediction.

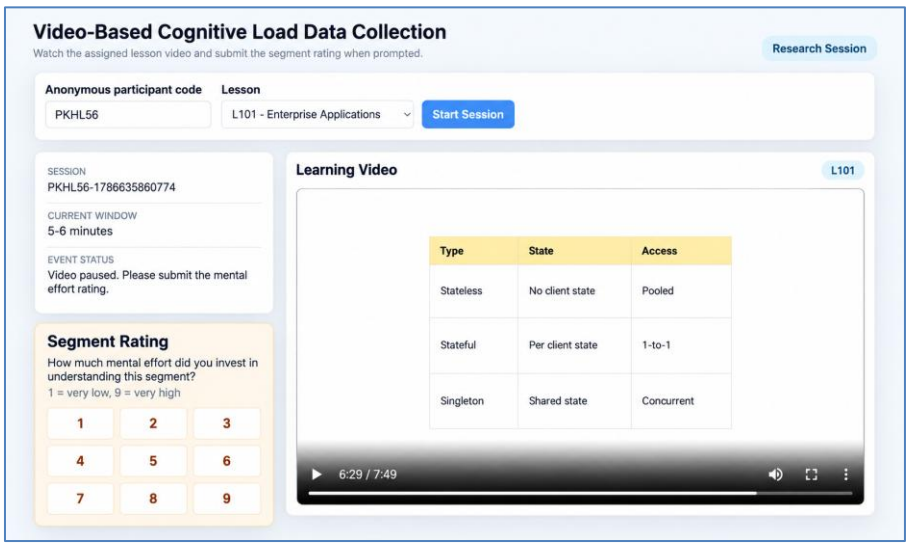


Figure 1: Video-based collection interface and segment-rating prompt.

3.4 Measures and evaluation metrics

The main reference measure is the Paas mental effort rating, grouped into five cognitive load levels. The model uses four event counts and two time measures as inputs.

Pause frequency counts confirmed video pauses. Video navigation counts forward seeks. Rewatch count records backward seeks. Playback-rate change count records adjustments to the video speed. Idle duration measures accumulated inactivity in seconds. Time on content measures effective observation time, although its calculation differs between the frontend and raw-event processing routes. *Table 1* defines the six inputs used by the deployed model.

Accuracy measures the percentage of observation windows assigned to the correct cognitive load class. Precision measures how often predictions for a particular class are correct. Recall measures how many windows belonging to that class are correctly identified. The F1-score combines precision and recall into a single measure.

Macro precision, macro recall, and macro F1 are calculated by averaging the results across the five classes, giving each class equal importance. Weighted F1 gives greater importance to classes with more test windows. Balanced accuracy averages recall across all five classes. Reporting both accuracy and macro metrics helps assess overall performance without allowing the most common class to dominate the results.

The confusion matrix shows how predictions are distributed across the five cognitive load classes. It identifies correctly classified windows and shows which classes are confused with one another. This provides a more detailed view of model performance than accuracy alone. Software tests assess whether the service functions correctly, while API response time measures how quickly it returns a result.

Table 1:Six-feature inference contract

Feature	Operational definition
pause_frequency	Count of confirmed video pauses
navigation_count_video	Count of forward seeks in current extraction
rewatch_segments	Count of backward seeks
playback_rate_change	Count of speed adjustments
idle_duration_video	Accumulated idle seconds
time_on_content	Effective time in seconds; route definitions require alignment

3.5 Data analysis

The dataset was divided into training and testing sets using an 80/20 participant-level split. All observation windows from a participant were assigned to the same set, preventing overlapping between training and testing. This approach reduces data leakage from repeated observations of the same learner. Since participants contributed different numbers of windows, the proportion of test windows could differ from 20%. The final evaluation included 293 test windows.

Logistic Regression, Decision Tree, XGBoost, and Random Forest were compared using accuracy, macro precision, macro recall, and macro F1-score. The deployed prediction pipeline combines median imputation with a Random Forest classifier. Median imputation replaces missing feature values with the median values learned during training. The classifier uses 200 trees, a maximum depth of 16, a minimum of 10 samples to split a node, balanced class weights, and a random state of 42.

To support reproducibility, the dataset version, participant assignments, preprocessing steps, and model settings should be documented. Imputation and model tuning should use training data only, with the testing set reserved for final evaluation. The available evidence identifies the deployed model configuration and evaluation results but does not fully document the tuning process or preprocessing settings for every comparison model.

3.6 Ethics privacy security and responsible AI

Participation was voluntary, and anonymous participant codes were used to link interaction records. Names, passwords, and other personal identifying information were not collected in the behavioral dataset. Participation did not affect academic marks or course evaluation. These measures helped protect participant privacy and reduce pressure to participate.

Signed consent records should be stored separately from the behavioral dataset and excluded from publicly shared research documents. This thesis describes the data collection instrument without reproducing participant codes, signatures, or identifiable photographs. Data retention arrangements and formal ethics approval details should be documented before final submission.

The reliability gate skips predictions when a session contains insufficient learning activity. However, passing this check does not guarantee an accurate estimate of mental effort. Predictions and explanations should support educational review rather than determine grades or diagnose learners. Before wider deployment, the system's access controls, cross-origin settings, and data protection measures should be reviewed.

3.7 Reproducibility and Research Artefacts

The available research artefacts include the proposal, data-collection introduction materials, consent form, project manuscript, model evidence report, CSV data exports, saved model bundle, database schema, frontend integration files, and API tests. The saved model bundle records six input features, five class labels, and the use of scikit-learn version 1.8.0. The evidence screenshots also document the model comparison, overall performance metrics, class-level results, feature importance, and confusion matrix.

Appendix A also records a difference between the dataset described in the study and a separate working data export. This does not prevent examination of the implemented system, but the limitation should be clearly acknowledged when evaluating the validity and reproducibility of the model.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The service must accept raw interaction events and precomputed feature windows while maintaining their association with the correct learner and session. It must predict one of five cognitive load levels and return a skipped result when there is insufficient learning activity. Suitable predictions must be stored for later analysis, and the six behavioral features must be sent to the LIME service for explanation. The service must also provide recent cognitive load trends and information about its operational status. *Table 2* summarizes these functional requirements and their implementation.

The non-functional requirements include consistent feature ordering, traceable results, timely responses, maintainability, and appropriate handling of dependency failures. Saved predictions are recorded in CSV format, with a fallback file used when the main file is locked. Database connection failures are logged. However, raw event extraction and trend analysis require access to the stored database records. These requirements guide the design, while further testing is needed to assess performance under different operating conditions.

Table 2:Component requirements and implementation evidence

Requirement	Implementation evidence
Event and window ingestion	POST /events/raw; POST /feature-windows
Five-class inference	POST /predict; POST /predict/from-raw
Reliability handling	not_reliable response with null class and reason
Prediction traceability	MySQL feature_windows and prediction_logs; CSV export
Explainability handoff	LIME HTTP dispatch and dispatch records
Temporal analysis	GET student/lesson load-trend
Operational inspection	GET /health; model and database metadata

4.2 Component architecture within the complete solution

The browser records video interactions and calculates behavioral statistics for each completed two-minute window. These data are sent through the Express API gateway to the FastAPI prediction service. The service checks whether the window contains sufficient learning activity, arranges the six features in the required order, and passes them to the trained model. It then returns the predicted cognitive load level and confidence score. When saving is requested, the service stores the feature window and prediction in MySQL, records the result in a CSV file, and attempts to send the features to LIME for explanation.

Figure 2 shows the cognitive load component within the complete learning platform, including the path followed when a window is skipped. The explainability and content adaptation components use their outputs to provide further learning support. They operate separately from the cognitive load classifier. The database stores raw interaction events, feature windows, prediction results, and records of attempts to send features to the explainability service.

The service also accepts raw interaction events directly. These events are stored and combined into complete two-minute windows that can be sent automatically to LIME. This process differs from the frontend's direct prediction request. Automatic feature dispatch does not create a cognitive load prediction log, while saved predictions provide the records used for trend analysis.

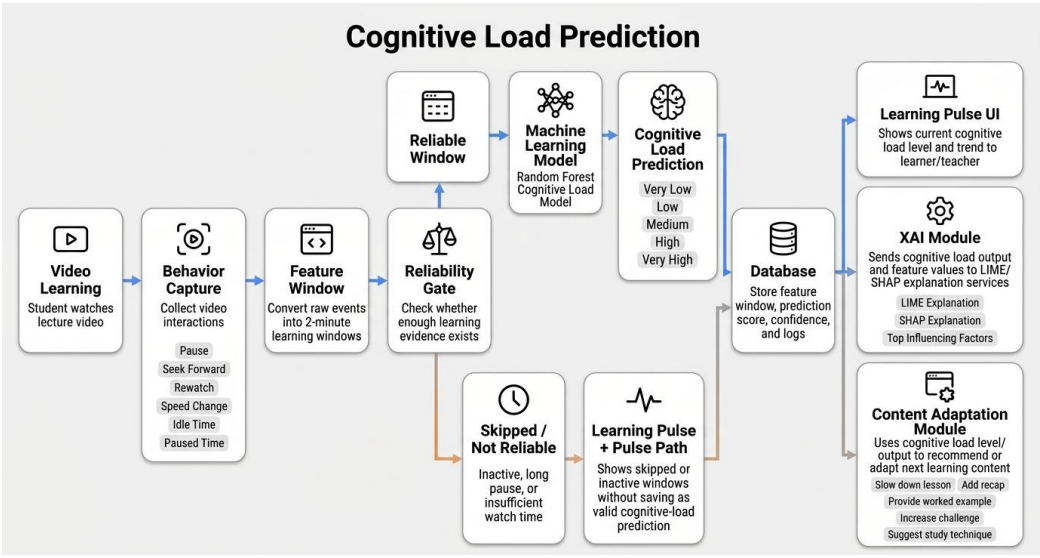


Figure 2:Cognitive load prediction workflow within the shared platform.

4.3 Design alternatives and decisions

The component uses video interaction data rather than physiological measurements to support non-intrusive monitoring through a standard web browser. Two-minute observation windows provide time to collect behavioral evidence and match the interval used for Paas ratings. Predictions are generated after each completed window. The recorded average API response time of 80 ms measures request processing, while the two-minute interval determines how frequently predictions become available.

Several classifiers were compared to assess their ability to capture relationships between behavioral features and cognitive load levels. Random Forest was selected because it achieved the strongest performance in the comparison. The deployed pipeline uses median imputation to handle missing numerical values.

A rule-based reliability gate skips windows with insufficient learning activity instead of assigning a prediction to every window. Cognitive load trends are identified using predefined rules applied to recent predictions. This approach makes the calculations easy to inspect without requiring a separate temporal model. However, the resulting risk summaries require further evaluation to establish their predictive validity.

4.4 Detailed design

Each prediction request includes learner and lesson identifiers, an optional session identifier, the window number, start and end times, and the six behavioral features. It can also include paused duration and a flag indicating whether the result should be saved. The service returns a cognitive load score from 1 to 5, the corresponding label, and the prediction time. Raw interaction records support events such as playing, pausing, seeking, changing playback speed and inactivity

Before prediction, the reliability gate checks whether the window contains sufficient learning activity. A window is skipped when:

1. Time on content is less than 60 seconds.
2. Paused duration is at least 60 seconds, with no forward navigation, rewatches, or playback-speed changes.

3. Idle duration is at least 60 seconds, with no more than one recorded interaction.

For raw event processing, the service includes events occurring at or after the window start and before the window end. This prevents an event at the boundary from being counted in two consecutive windows. It counts the relevant interactions and calculates idle duration from inactivity events. The raw-event route calculates time on content by subtracting idle duration from the window duration. The frontend calculation also subtracts paused duration. These calculations should be aligned with the feature definition used during model training.

Trend analysis retrieves recent saved windows. It examines up to four recent valid scores to identify rising, decreasing, stable, or fluctuating cognitive load. When fewer than two valid scores are available, it returns an insufficient data result. The risk summary combines the current load score, its trend, and consecutive high load predictions. These summaries describe patterns in model predictions and require further validation before being interpreted as indicators of learner difficulty.

4.5 Standards and professional practice

The component exchanges data through HTTP requests using JSON. Pydantic schemas define the expected inputs, while consistent feature names and database identifiers help trace records to the correct learner, lesson, and session. Timestamps use ISO format, and prediction creation times are recorded in UTC. Parameterised database queries keep input values separate from SQL statements. The saved model includes feature and class metadata to support consistent use across services. Model explanations are interpreted as statistical contributions rather than proof of causation. Dataset versions and evaluation limitations are also documented to support transparency and reproducibility.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development of environment and technology stack

The cognitive load component is developed in Python. FastAPI defines the API endpoints, while Uvicorn runs the service. Pydantic validates request data, and pandas and NumPy support numerical data processing. The machine learning pipeline uses scikit-learn, with joblib used to load the saved model. The model metadata records scikit-learn version 1.8.0. MySQL stores interaction events, feature windows, prediction results, and explainability dispatch records through mysql-connector-python.

The frontend uses React and sends requests through Axios or Fetch API. An Express gateway forwards these requests to the appropriate service using http-proxy-middleware.

For local development, the prediction service runs on port 8021 within its Python virtual environment. The API gateway runs on port 4000. The LIME service is accessed through a configurable endpoint, which defaults to port 8110. These settings support local development and testing.

5.2 Component implementation

The model is loaded once when the service starts. The loader supports either a model bundle containing metadata or a standalone model object. If loading fails, the service raises a startup error. The deployed pipeline combines median imputation with a Random Forest classifier. Inference uses one worker, and the six behavioral features are supplied in the same order expected by the trained model.

Before running the model, the prediction service checks whether the observation window contains sufficient learning activity. A window that fails this check returns a **not_reliable** status, no predicted class or score, zero confidence, and a reason for skipping. No prediction result is saved or sent through the normal LIME dispatch process. For an accepted window, the model generates a class prediction and class probabilities. The response includes the cognitive load label, score, and highest class

probability, rounded to two decimal places. This confidence value represents the model's output and has not been independently calibrated.

The /predict endpoint saves results only when **save_result** is enabled. The frontend enables this option, while /predict/from-raw extracts features from stored events and requests saving through the same prediction service. CSV records contain the behavioral features, predicted label and score, confidence, and creation time. MySQL also preserves the session associations needed for session-based analysis. The /xai/data endpoint retrieves CSV prediction records, while cognitive load trend analysis uses the feature windows and prediction logs stored in MySQL.

5.3 Integration with the other components

The frontend video player and course view request a prediction after each completed two-minute window. They track behavioral statistics and session identifiers and use request controls to reduce duplicate submissions. Confirmed pauses are distinguished from temporary pauses caused by seeking. Each request includes the six model features and paused duration. Windows with insufficient activity are displayed as skipped rather than assigned a cognitive load level.

Saved reliable windows are sent to LIME with learner and session identifiers, window boundaries, and the six behavioral features. Dispatch records identify successful submissions to prevent repeated processing. Failed attempts are recorded with an error message.

The predicted cognitive load level is also passed to the content adaptation component. This component uses the prediction, together with available learner information, to adjust the presentation and complexity of learning content.

After an accepted prediction, the frontend retrieves recent cognitive load trends for the learner, lesson, and session. The prediction data also supports teacher summaries, study technique recommendations, lesson planning, and adaptive questions. The individual component provides the prediction data and service interfaces, while the effectiveness of these learning-support functions requires separate evaluation.

5.4 Verification and software testing

The component was verified using an automated unit test suite. All 15 tests passed. The tests covered feature extraction, idle intervals, completed-window dispatch, duplicate dispatch handling, prediction output normalization, reliability checks, storage value conversion, and cognitive load trend analysis. *Table 3* summarizes the test coverage.

Controlled inputs and simulated dependencies were used where appropriate to test specific functions. The results confirm that these functions behaved as expected under the conditions tested. Further integration testing is needed to assess database connectivity, concurrent requests, browser event logging, and dependency failures.

An additional verification task is to compare raw-event extraction with frontend feature calculation. The same interaction sequence should be processed through both routes to check consistency in pause counts, idle duration, navigation, and effective watch time.

Table 3:Component test coverage

Test module	Tests	Coverage
Feature extraction	2	Event conversion and open idle interval
LIME dispatch	3	Session requirement, completed windows, duplicate success
Prediction service	7	Output normalisation, reliability, storage helpers
Trend analysis	3	Rising/high risk, decreasing, stable low load
Total	15	All passed in the executed unittest suite

5.5 Deployment and operation

The service runs locally through the PowerShell launcher start-api.ps1, which starts Uvicorn within the component's Python virtual environment. The /health endpoint provides database connection status, model information, and supported interaction event types. Database settings and the LIME service endpoint can be configured through environment variables.

When MySQL is unavailable, the service can still generate predictions from suitable precomputed features. However, raw event extraction, stored trend analysis, and dispatch tracking require access to database records. If the main CSV file is locked, predictions are written to a fallback file. Requests to the LIME service have a timeout of 15 seconds, so delays in that service can affect the response time of prediction requests that include saving and dispatch.

6 RESULTS AND VALIDATION

6.1 Research results

Table 4 compares the performance of Logistic Regression, Decision Tree, XGBoost, and Random Forest. Random Forest achieved the highest accuracy of 79.52% and macro F1-score of 79.55%. XGBoost achieved the next highest performance, with 75.77% accuracy and 75.95% macro F1-score. These results supported the selection of Random Forest for deployment.

The selected model achieved 79.48% balanced accuracy, 79.87% macro precision, 79.48% macro recall, and 79.56% weighted F1-score. Of the 293 test windows, 233 were classified correctly, giving an accuracy of 79.52%. *Figure 3* presents the confusion matrix, while *Table 5* summarizes recall for each cognitive load class.

The model correctly identified 57 of 60 Very Low windows, achieving 95.00% recall. Medium achieved 84.48% recall, with 49 of 58 windows correctly classified. Low and High each achieved 69.49% recall, with 41 of 59 windows correctly classified. Very High achieved 78.95% recall, with 45 of 57 windows correctly classified.

Figure 4 presents the model's feature importance ranking. Idle duration had the highest importance, followed by pause frequency, time on content, navigation count, rewatch count, and playback-rate changes. These values indicate how strongly the fitted model used each feature when making predictions. They do not establish that a particular behavior directly causes higher cognitive load.

Table 4:Reported classifier comparison with percentage metrics

Model	Accuracy	Precision	Recall	F1
Logistic Regression	66.89	66.02	66.83	66.23
Decision Tree	67.58	66.98	67.58	67.15
XGBoost	75.77	76.50	75.71	75.95
Random Forest	79.52	79.87	79.48	79.55

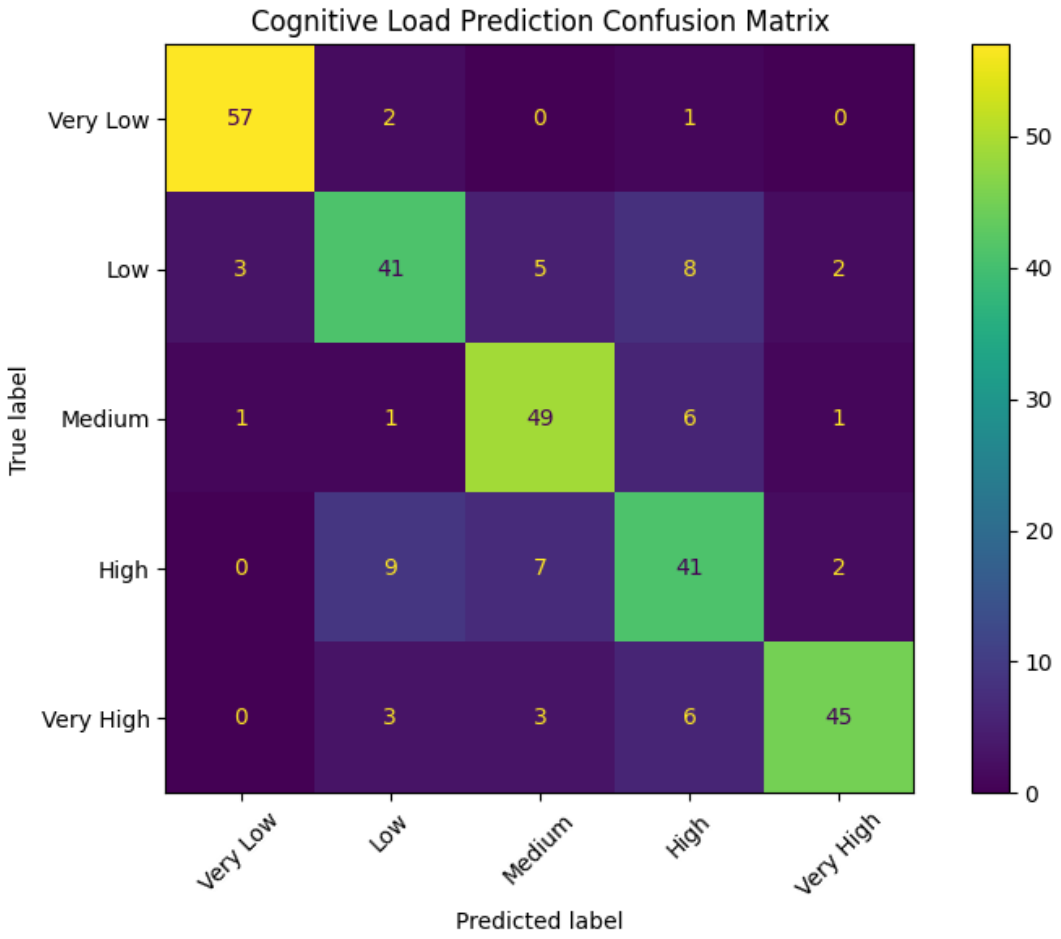


Figure 3:Reported Random Forest confusion matrix on 293 held-out windows.

Table 5:Class recall calculated from the reported matrix

Class	Support	Correct	Recall (%)
Very Low	60	57	95.00
Low	59	41	69.49
Medium	58	49	84.48
High	59	41	69.49
Very High	57	45	78.95
Total	293	233	Accuracy 79.52

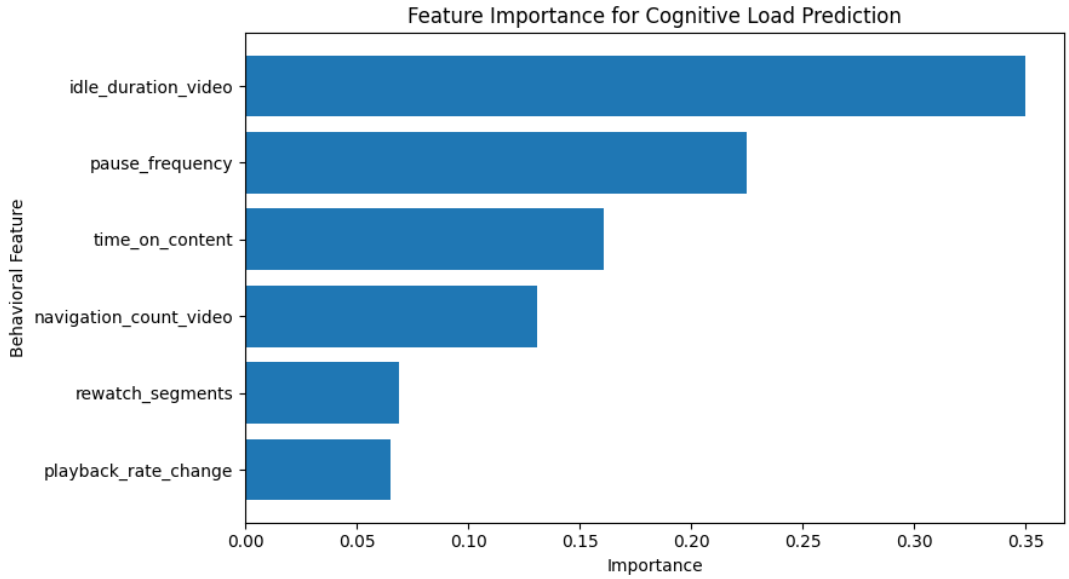


Figure 4: Feature importance ranking in the supplied model evaluation.

6.2 Product and technical results

The implemented service supports predictions from both raw interaction events and precomputed behavioral features. It returns a cognitive load prediction for windows that pass the reliability checks and a skipped result for windows with insufficient activity. The service also stores feature windows and prediction results, sends behavioral features to LIME, and provides recent cognitive load trends. All 15 component tests passed. The saved model contains the six-feature prediction pipeline and the five cognitive load classes.

A local API benchmark recorded a total response time of 800 ms across ten requests, giving an average of 80 ms per request. This result demonstrates responsiveness under the tested local conditions. Further testing is needed to assess performance with concurrent users, larger request volumes, and delays from supporting services.

Predictions become available after each complete two-minute observation window. The observation interval determines how frequently cognitive load is estimated, while API response time measures how quickly a request is processed. Reliability indicators and trend display help users interpret the results. Database operations and communication with the explanation service may also affect the overall response time.

6.3 User or stakeholder validation

Data collection involved undergraduate computing students participating in laboratory and remote settings. Participants watched instructional videos, interacted naturally with the learning interface, and provided Paas mental effort ratings after each two-minute segment.

6.4 Integration results

The integrated interface displays cognitive load predictions alongside the behavioral feature values used by the model. Demonstrated outputs include Very Low, Medium, High, and Very High levels, together with a waiting or skipped state when learning activity is insufficient. The interface also presents recent cognitive load trends and the current learning state.

The frontend submits prediction requests with result saving enabled and retrieves trend history for the relevant session. The service stores the feature window and prediction before attempting to send the behavioral features to LIME. Automated tests verify the selection of completed windows and the prevention of repeated dispatch after a successful submission.

These demonstrations and tests provide evidence of integration between the frontend, prediction service, data storage, and explainability workflow. Further end-to-end testing is needed to verify the complete process under live operating conditions, including database writes and responses from downstream services.

6.5 Summary of findings

The study demonstrated five-class cognitive load prediction using six video interaction features. Random Forest achieved 79.52% test accuracy and a macro F1-score of 79.55%. The deployed pipeline combines median imputation with the selected classifier. Very Low achieved the highest recall, while distinguishing between Low and High remained more challenging.

The implemented service provides reliability checks, skipped-window handling, prediction storage, LIME integration, and cognitive load trend analysis. All 15 component tests passed, and a local benchmark recorded an average response time of 80 ms across ten requests.

These findings support the feasibility of the component within the studied learning context. Further evaluation is needed to assess performance across different institutions and learner populations, the accuracy of confidence and risk estimates, and the effects of adaptive support on learning outcomes.

7 DISCUSSION

7.1 Interpretation of findings

The findings suggest that the six video interaction features provide useful information for estimating students' self-reported mental effort. Random Forest performed better than the other evaluated classifiers, suggesting that combinations of behavioral features can help distinguish cognitive load levels. However, these relationships were evaluated within a specific student population and two instructional videos. The model estimates mental effort from behavior rather than measuring a learner's cognitive state directly.

Predictions should therefore be considered alongside the recorded behaviors, recent trends, and teacher judgement when deciding whether to adapt learning content. The reliability gate skips windows with insufficient activity, but passing this check does not guarantee that a prediction is correct.

7.2 Comparison with literature baselines and existing solutions

The component uses video interaction data to predict five cognitive load levels rather than only describing learner activity. This approach is consistent with Merkt et al.'s findings that pausing can relate to comprehension difficulty and learning regulation [3]. However, pauses may also reflect useful activities such as note-taking. Combining six behavioral features allows the model to consider several aspects of learner activity instead of relying on pauses alone.

Unlike the physiological approaches discussed by Cai and Demmans Epp and by Ajayi and Avksentyeva, this component does not require eye trackers or wearable sensors [4], [5]. It uses interactions recorded through a standard browser, making data collection easier to integrate into an online learning platform.

The gateway and MySQL architecture provide a simpler local implementation than the distributed clickstream systems discussed by Gayakwad et al. [6].

7.3 Research contribution

This research contributes an approach for predicting five cognitive load levels from video interaction data. It links behavioral observations with Paas mental-effort ratings collected after each two-minute segment. Several classifiers were evaluated using separate participant groups for training and testing, and class-level results were analyzed to identify prediction strengths and common errors.

The study also integrates the selected model with reliability checks and cognitive load trend analysis. The trend module summarizes changes in recent predictions without requiring a separate trained model.

The contribution lies in combining established methods into an integrated prediction workflow for video-based online learning. Random Forest, the Paas scale, and event aggregation are existing techniques; the study applies them within a service that supports further explanation and content adaptation. The effect of this approach on learning outcomes remains a topic for future evaluation.

7.4 Product contribution

The product contribution is an independent prediction service with a clear feature contract and explicit output states. Learner, lesson, session, and window information support traceability. Reliable predictions can feed explanation and adaptation, while skipped windows communicate the absence of sufficient evidence. Saved history supports a temporal learner interface and teacher review.

The component is useful to the group platform because it transforms low-level events into a common learner-state representation. Its separation from downstream generation permits independent maintenance and testing. The broader platform's teacher review controls provide an additional human interpretation layer, but they cannot replace careful validation of the underlying measurements.

7.5 Limitations and threats to validity

The study uses self-reported Paas ratings as the reference measure for cognitive load. These ratings may be influenced by prior knowledge, emotions, interruptions, and individual interpretations of the scale. Grouping nine rating values into five classes reduces the detail available in the original responses. The ratings also do not

separately measure intrinsic and extraneous cognitive load. Requesting a rating every two minutes may affect participants' natural viewing behavior.

The participant-level split reduces data leakage by keeping each participant's observations within either training or testing. However, complete split assignments and preprocessing records are needed to reproduce the evaluation. Differences between development exports and the final study dataset also require clear version tracking.

The study involved undergraduate computing students and two instructional videos, which limit generalization to other populations and subjects. Laboratory and remote conditions may influence behavior differently, but performance was not evaluated separately for these groups. Confidence intervals, repeated participant-level evaluations, and significance tests would strengthen the statistical assessment. Such analyses should account for repeated observations from the same participant.

Implementation limitations include differences in watch-time calculations between the frontend and raw-event processing routes, the handling of idle periods across window boundaries, and limited testing of concurrent requests.

7.6 Implications

For teachers, cognitive load predictions provide information about individual learning segments that can help identify material requiring review or additional support. Predictions should be considered alongside recorded behaviors, recent trends, and teacher judgement.

For learners, the interface displays their estimated cognitive load and distinguishes windows with insufficient activity from those predicted as low load. This helps learners understand how the system interprets their interactions.

For developers, feature calculations must remain consistent between model training and deployment. Clear definitions of pauses, navigation, idle duration, and effective watch time help ensure that the model receives comparable inputs.

For future research, participant-level evaluation with documented dataset versions would improve reproducibility. Intervention studies are also needed to examine

whether adaptive support improves learning outcomes and mental effort. Avoiding specialized hardware makes the approach easier to deploy, but its fairness and effectiveness across different learner groups requires further evaluation.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This thesis developed and evaluated the cognitive load prediction component of an explainable adaptive learning platform. The component uses six video interaction features collected over two-minute windows to predict five cognitive load levels based on grouped Paas mental-effort ratings. The study involved 171 undergraduate computing students, comprising 108 laboratory and 63 remote participants, and produced 1,420 valid observation windows. Random Forest achieved the strongest performance among the evaluated classifiers, with 79.52% accuracy and a macro F1-score of 79.55%.

The deployed pipeline combines median imputation with a 200-tree Random Forest classifier. The API provides reliability checks, prediction storage, LIME integration, and cognitive load trend analysis. Predicted load levels are also supplied to the content adaptation component. All 15 component tests passed, supporting the functionality of the tested service operations.

The findings demonstrate the feasibility of estimating self-reported mental effort from video behavior within the studied context. Further evaluation is needed to assess generalization to other learner populations and determine whether the resulting adaptive support improves learning outcomes.

8.2 Achievement of objectives

1. **Objective 1 - Establish a consent-based data collection procedure:** A voluntary collection process was implemented using anonymous participant identifiers, two instructional videos, and Paas mental-effort ratings collected after each segment.
2. **Objective 2 - Extract six consistent behavioral features:** Two-minute window aggregation and the six-feature model input structure were implemented. Further work is needed to align watch-time calculations between the frontend and raw-event processing routes.

3. **Objective 3 - Compare classifiers and select a deployable model:** Candidate classifiers were evaluated using a participant-level split, and the selected Random Forest pipeline was saved for deployment. Complete training and split records are needed to reproduce the evaluation.
4. **Objective 4 - Integrate prediction and supporting services:** API endpoints were implemented for prediction, reliability checks, data storage, and feature dispatch to the explainability service.
5. **Objective 5 - Provide temporal summaries and verify the service:** Cognitive load trend and risk summaries were implemented, and all 15 component tests passed. Further evaluation is needed to determine whether the risk summaries predict future cognitive overload.

8.3 Individual research and product contributions

The individual research contribution is a behavioral modelling approach that links video interactions with segment-level Paas ratings to predict five cognitive load levels. It includes the evaluation of candidate classifiers and analysis of their performance and prediction errors.

The product contribution is a reusable cognitive load prediction service integrated with the video learning interface. The service provides reliability checks, prediction storage, explainability dispatch, and recent cognitive load trends. It also supplies predicted load levels to the content adaptation component and skips windows with insufficient learning activity.

These contributions support the wider platform's cognitive style detection, explainability, knowledge processing, and teacher-reviewed learning support. The individual work focuses on cognitive load prediction and its integration with these shared components.

8.4 Future directions

Future work should improve reproducibility by preserving the final dataset, participant assignments, data cleaning records, training scripts, and preprocessing settings. Feature calculations should also be aligned across the frontend and server so that the same interaction data produce consistent model inputs.

Model evaluation should include repeated participant level testing, confidence intervals, and assessment of confidence scores. Additional measures could examine how far incorrect predictions differ from the true cognitive load level.

Further studies should involve more videos, subjects, institutions, and devices to assess generalizability. Laboratory and remote participants should be evaluated separately to examine differences between collection settings. Alternative temporal methods could also be compared with the current trend and risk rules.

Technical evaluation should assess concurrent requests and end-to-end response times, including database operations and explainability processing. Finally, a controlled study conducted over a longer period should investigate whether teacher-reviewed adaptive support improves understanding, knowledge retention, and mental effort without unnecessarily interrupting learning.

REFERENCES

[1] J. Sweller, “Cognitive load theory,” in Psychology of Learning and Motivation, vol. 55, 2011, pp. 37–76.

[2] F. G. W. C. Paas, “Training strategies for attaining transfer of problem-solving skill in statistics: A cognitive-load approach,” Journal of Educational Psychology, vol. 84, no. 4, pp. 429–434, 1992.

[3] M. Merkt, A. Hoppe, G. Bruns, R. Ewerth, and M. Huff, “Pushing the button: Why do learners pause online videos?,” Computers & Education, vol. 176, Art. no. 104355, 2022, doi: 10.1016/j.compedu.2021.104355.

[4] M. Cai and C. Demmans Epp, “Modeling cognitive load and affect to support adaptive online learning,” in Proc. 15th Int. Conf. Educational Data Mining, Doctoral Consortium, Durham, United Kingdom, 2022, pp. 799–804.

[5] I. H. Ajayi and E. Avksentyeva, “Machine learning methods for predicting cognitive load in eLearning environments using eye tracking data,” Pakistan Journal of Life and Social Sciences, vol. 22, no. 2, pp. 19926–19937, 2024, doi: 10.57239/PJLSS-2024-22.2.001458.

[6] M. Gayakwad et al., “Real-time clickstream analytics with Apache,” Journal of Electrical Systems, vol. 20, no. 2, pp. 1600–1608, 2024.

[7] C. Lange, “The relationship between e-learning personalisation and cognitive load,” Open Learning: The Journal of Open, Distance and e-Learning, published online Dec. 26, 2021, doi: 10.1080/02680513.2021.2019577.

[8] L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] M. T. Ribeiro, S. Singh, and C. Guestrin, “Why should I trust you?: Explaining the predictions of any classifier,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144.

[10] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems, vol. 30, 2017.

[11] I. H. Ajayi and E. Avksentyeva, “Machine learning methods for predicting cognitive load in eLearning environments using eye tracking data,” Pakistan Journal of Life and Social Sciences, vol. 22, no. 2, pp. 19926–19937, 2024, doi: 10.57239/PJLSS-2024-22.2.001458.

GLOSSARY

- Cognitive load estimate: A model category inferred from behaviors and referenced to grouped self-reported mental effort.
- Feature window: A fixed observation interval whose behavioral events are aggregated into model inputs.
- Participant leakage: Information shared between training and testing through repeated observations from the same participant.
- Reliability gate: A rule that skips inference when the observation contains insufficient activity; it does not guarantee that an accepted prediction is correct.
- Reported result: An experimental value preserved from the supplied research record, distinct from a result regenerated during thesis preparation.

APPENDICES

Appendix A Research instruments and evaluation evidence

A.1 Collection instrument

Participants watch a selected educational video and may pause, rewind, navigate, or change the playback speed naturally. After each two-minute segment, they rate the mental effort required to understand that segment on a nine-point Paas scale, where 1 indicates very low effort and 9 indicates very high effort. These ratings are grouped into five cognitive load classes:

- **Very Low:** 1–2
- **Low:** 3–4
- **Medium:** 5
- **High:** 6–7
- **Very High:** 8–9

Participation is voluntary, and interaction data are recorded using anonymous participant codes. Signed consent records are stored separately from the behavioral dataset and are not included in this thesis.

Participants either use assigned code and lesson or generate a code and select a lesson through the collection interface. Both procedures use the same rating scale and class mapping. The anonymous code links each participant's observation windows and supports participant-level separation during model evaluation. It is not used as an input to the classifier.

A.2 Study figures and development export reconciliation

The study involved 171 participants, comprising 108 laboratory and 63 remote participants, and produced 1,420 valid two-minute observation windows. These figures are used consistently throughout the thesis. The reported confusion matrix contains 293 testing windows.

A separate development file, *Final_cognitive_load_dataset.csv*, contains 1,471 rows, 341 distinct participant codes, and 392 session identifiers. Another dataset summary

records 461 distinct identifiers. These counts refer to different development records and are not used to determine the study's participant total. The relationship between these records and the final training dataset requires further documentation.

Inspection of the 1,471-row export identified 15 duplicate rows and 58 rows with at least one missing model feature. The missing-value counts were seven for pause frequency, nine for navigation, twelve for rewatches, seven for playback-rate changes, fourteen for idle duration, and nine for time on content. All recorded Paas ratings followed the specified five-class mapping. These observations describe the development export but do not establish how individual rows were processed in the reported experiment.

Appendix B Academic integrity and AI use disclosure

1. **Tools used:** OpenAI ChatGPT and Codex were used to help draft and structure the thesis, explain technical concepts, and improve academic writing. React, Python, FastAPI, pandas, NumPy, scikit-learn, and MySQL were used to develop, evaluate, and integrate the Cognitive Load Prediction System.
2. **Materials provided:** Inputs included the proposal, institutional thesis template, collection slides, consent form, supporting literature, team research paper, component evidence report, and local implementation.
3. **Instructions followed:** The main instructions were to develop the individual thesis using the proposal and template, explain the data collection process, follow IEEE referencing conventions, and incorporate relevant findings from the team research.
4. **Contribution boundaries:** Individual contributions were distinguished from team contributions. Reported experimental results were also separated from evidence obtained through software inspection.
5. **Verification performed:** Checks included source-text extraction, inspection of evaluation figures, confusion-matrix calculations, saved model parameter inspection, a development CSV audit, execution of 15 existing API tests, and document layout review.